Patent Document 4

SVMDex — Clustering Using SVMs for Indexing in High-Dimensional Spaces (Morpho Software Confidential)

Edward Chang
Morpho Software Inc.
3944 State Street, #340
Santa Barbara, CA 93105
echang@db.stanford.edu

March 22, 2001

Abstract

In this paper we present a clustering and indexing paradigm (called SVMDex) for high-dimensional search spaces. The scheme is designed for approximate similarity searches, where one wishes to find many of the data points near a target point, but where one can tolerate missing a few near points. For such searches, our scheme can find near points with high recall in very few IOs and perform significantly better than other approaches. SVMDex has two phases: forming clusters and then building indexes. In the clustering phase, SVMDex finds clusters using Support Vector Machines (SVMs). In the indexing phase, it computes the centroids and support vectors for clusters and then build an index structure for locating nearby clusters given a query point. We analyze the tradeoffs involved in clustering and building such an index structure, and present extensive experimental results.

Keywords: approximate search, clustering, high-dimensional index, similarity search.

1 Introduction

Similarity search has generated a great deal of interest lately because of applications such as similar text/image search and document/image copy detection. These applications characterize objects (e.g., images and text documents) as *feature vectors* in very high-dimensional spaces [30, 40]. A user submits a query object to a search engine, and the search engine returns objects that are similar to the query object. The degree of similarity between two objects is measured by some distance function between their feature vectors. The search is performed by returning the objects that are *nearest* to the query object in high-dimensional spaces.

Nearest-neighbor search is inherently expensive, especially when there are a large number of dimensions. First, the search space can grow exponentially with the number of dimensions.

Second, there is simply no way to build an index on disk such that *all* nearest neighbors to *any* query point are physically adjacent on disk. (We discuss this "curse of dimensionality" in more detail in Section 2.) Fortunately, in many cases it is sufficient to perform an *approximate search* that returns many but not all nearest neighbors [2, 17, 15, 27, 30, 31]. (A feature vector is often an approximate characterization of an object, so we are already dealing with approximations anyway.) For instance, in content-based image retrieval [23, 19, 42] and document copy detection [9, 13, 20], it is usually acceptable to miss a small fraction of the target objects. Thus it is not necessary to pay the high price of an exact search.

In this paper we present an indexing scheme that combines clustering and indexing to achieve approximate similarity search with high efficiency. We call this approach *SVMDex* (Support Vector Machine clustering for inDEXing). Under SVMDex, the dataset is first partitioned into "similar" clusters by applying SVMs. To improve IO efficiency, each cluster is then stored in a sequential file, and a mapping table is built for indexing the clusters. To answer a query, clusters that are near the query point are retrieved into main memory. SVMDex then ranks the objects in the retrieved clusters by their distances to the query object, and returns the top, say k, objects as the result.

Both clustering and indexing have been intensively researched (we survey related work in Section 2), but these two subjects have been studied separately with different optimization objectives: clustering optimizes classification accuracy, while indexing maximizes IO efficiency for information retrieval. Because of these different goals, indexing schemes often do not preserve the clusters of datasets, and randomly project objects that are close (hence similar) in high-dimensional spaces onto a 2D plane (the disk geometry). This is analogous to breaking a vase (cluster) apart to fit it into the minimum number of small packing boxes (disk blocks). Although the space required to store the vase may be reduced, finding the boxes in a high-dimensional warehouse to restore the vase requires a great deal of effort.

In this study we show that by (1) taking advantage of the clustering structures of a dataset, and (2) taking advantage of sequential disk IOs by storing each cluster in a sequential file, we can achieve efficient approximate similarity search in high-dimensional spaces. In addition, we take advantage of the support vectors and hyperplanes generated by SVMs to achieve accurate search. These support vectors and hyperplanes, together with the centroids of clusters, provide both the center and boundary information of clusters so that SVMDex can find nearby clusters of a query point more accurately. We examine a variety of clustering algorithms on two different data sets to show that SVMDex works well when (1) a dataset can be grouped into clusters and (2) an algorithm can successfully find these clusters.

The contributions of this paper are as follows:

- We show how to apply SVMs to find clusters and how to generate support vectors and a hyperplane to characterize each cluster. The centroids and hyperplanes of clusters lead to a

simple index that performs very well for approximate similarity searches.

- We also evaluate SVMDex with different clustering algorithms (e.g., TSVQ), and compare SVMDex with other traditional approaches (e.g., tree structures) to understand the gains achievable by pre-processing data to find clusters.
- Finally, we compare SVMDex with PAC-NN [15], a latest approach for conducting approximate nearest neighbor search.

The rest of the paper is organized as follows. Section 2 discusses related work. Section 3 presents SVMDex and shows how a similarity query is conducted using our scheme. Section 4 presents the results of our experiments and compares the efficiency and accuracy of our approach with those of some traditional index structures. Finally, we offer our conclusions and discuss the limitations of SVMDex in Section 5.

2 Related Work

In this section we discuss related work in three categories:

1: Tree-like index structures for similarity search,

2: Approximate similarity search, and

3: Indexing by clustering.

2.1 Tree-like Index Structures

Many tree structures have been proposed to index high-dimensional data (e.g., R*-tree [3, 24], SS-tree [48], SR-tree [29], TV-tree [33], X-tree [6], M-tree [16], and K-D-B-tree [38]). A tree structure divides the high-dimensional space into a number of subregions, each containing a subset of objects that can be stored in a small number of disk blocks. Given a vector that represents an object, a similarity query takes the following three steps in most systems [21]:

1. It performs a *where-am-I* search to find in which subregion the given vector resides.

2. It then performs a *nearest-neighbor* search to locate the neighboring regions where similar vectors may reside. This search is often implemented using a *range* search, which locates all the regions that overlap with the search sphere, i.e., the sphere centered at the given vector with a diameter d.

3. Finally, it computes the distances (e.g., Euclidean, street-block, or L^∞ distances) between the vectors in the nearby regions (obtained from the previous step) and the given vector. The search result includes all the vectors that are within distance d from the given vector.

The performance bottleneck of similarity queries lies in the first two steps. In the first step, if the index structure does not fit in main memory and the search algorithm is inefficient, a

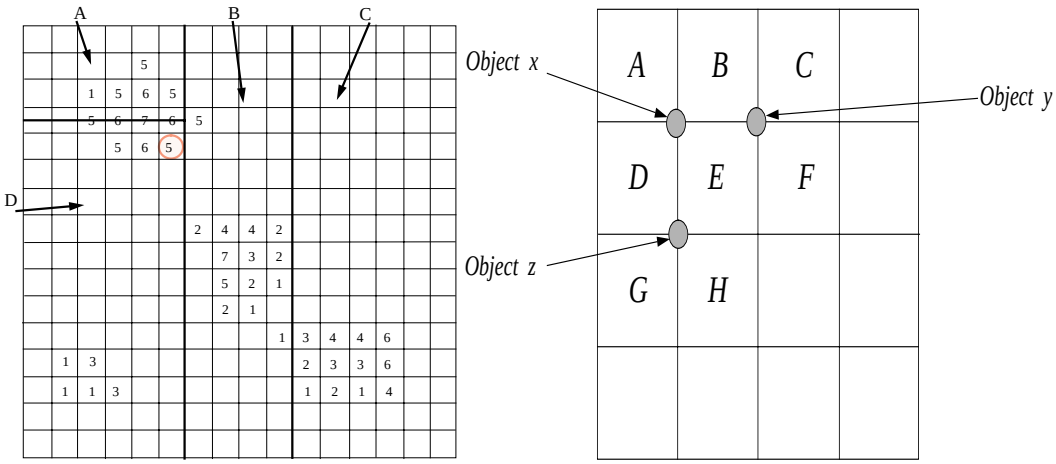
large portion of the index structure must be fetched from the disk. In the second step, the number of neighboring subregions can grow exponentially with respect to the dimension of the feature vectors. If D is the number of dimensions, the number of neighboring subregions can be on the order of $O(3^D)$ [21]. Roussopoulos et. al [39] propose the *branch-and-bound* algorithm and Hjaltason and Samet [25] propose the *priority queue* scheme to reduce the search space. But, when D is very large, the reduced number of neighboring pages to access can still be quite large. Berchtold et. al [5] propose the pyramid technique that partitions a high dimensional space into $2D$ pyramids and then cuts each pyramid into slices that are parallel to the basis of the pyramid. This scheme does not perform satisfactorily when the data distribution is skewed or when the search hypercube touches the boundary of the data space.

In addition to being copious, the IOs can be random and hence exceedingly expensive.[1] An example can illustrate what we call the *random-placement* syndrome faced by traditional index structures. Figure 1(a) shows a 2-dimensional Cartesian space divided into 16 equal stripes in both the vertical and the horizontal dimensions, forming a 16×16 grid structure. The integer in a cell indicates how many points (objects) are in the cell. Most index structures divide the space into subregions of equal points in a top-down manner. Suppose each disk block holds 40 objects. One way to divide the space is to first divide it into three vertical compartments (i.e., left, middle, and right), and then to divide the left compartment horizontally. We are left with four subregions A, B, C and D containing about the same number of points. Given a query object residing near the border of A, B and D, the similarity query has to retrieve blocks A, B and D. The number of subregions to check for the neighboring points grows exponentially with respect to the data dimension.

Furthermore, since in high-dimensional spaces the neighboring subregions cannot be arranged in a manner sequential to all possible query objects, the IOs must be random. Figure 1(b) shows a 2-dimensional example of this random phenomenon. Each grid in the figure, such as A and B, represents a subregion corresponding to a disk block. The figure shows three possible query objects x, y and z. Suppose that the neighboring blocks of each query object are its four surrounding blocks. For instance, blocks A, B, D and E are the four neighboring blocks of object x. If the neighboring blocks of objects x and y are contiguous on disk, then the order must be $CFEBAD$, or $FCBEDA$, or their reverse orders. Then it is impossible to store the neighboring blocks of query object z contiguously on disk, and this query must suffer from random IOs. This example suggests that in high-dimensional spaces, the neighboring blocks of a given object must be dispersed randomly on disk by tree structures.

Many theoretical papers (e.g., [2, 27, 30]) have studied the cost of an exact search, independent of the data structure used. In particular, these papers show that if N is the size of a dataset, D is the dimension, and $D >> log\ N$, then no nearest-neighbor algorithm can be

[1] To transfer 100 KBytes of data on a modern disk with 8-KByte block size, doing a sequential IO is more than ten times faster than doing a random IO. The performance gap widens as the size of data transferred increases.



(a) Clustering by indexing (b) Random placement

Figure 1: The shortcomings of tree structures.

significantly faster than a linear search.

2.2 Approximate Similarity Search

Many studies propose conducting approximate similarity search for applications where trading a small percentage of recall for faster search speed is acceptable. For example, instead of searching in all the neighboring blocks of the query object, study [47] proposes performing only the where-am-I step of a similarity query, and returning only the objects in the disk block where the query object resides. However, this approach may miss some objects similar to the query object. Take Figure 1(a) as an example. Suppose the query object is in the circled cell in the figure, which is near the border of regions A, B and D. If we return only the objects in region D where the query object resides, we miss many nearest neighbors in A.

Arya and Mount [2] suggest doing only ε-approximate nearest-neighbor searches, for $\varepsilon > 0$. Let d denote the function computing the distance between two points. We say that $p \in P$ is an ε-approximate nearest neighbor of q if for all $p' \in P$ we have $d(p,q) \leq (1+\varepsilon)d(p',q)$. Many follow-up studies have attempted to devise better algorithms to reduce search time and storage requirements. For example, Indyk and Motwani [27] and Kushilevitz et al. [31] give algorithms with polynomial storage and query time polynomial in $logn$ and d. Indyk and Motwani [27] also give another algorithm with smaller storage requirements and sublinear query time. Most of this work, however, is theoretical. The only practical scheme that has been implemented is the *locality-sensitive hashing* scheme proposed by Indyk and Motwani [27]. The key idea is to use hash functions such that the probability of collision is much higher for objects that are close to each other than for those that are far apart.

Approximate search has also been applied to tree-like structures. Recent work [15] shows that if one can tolerate $\varepsilon > 0$ relative error with a δ confidence factor, one can improve the performance of M-tree by 1-2 orders of magnitude.

Although an ε-approximate nearest-neighbor search can reduce the search space significantly, its recall can be low. This is because the candidate space for sampling the nearest neighbor becomes exponentially larger than the optimal search space. To remedy this problem, a follow-up study of [27] builds multiple *locality-preserving* indexes on the same dataset [26]. This is analogous to building n tree indexes on the same dataset, and each index distributes the data into data blocks differently. To answer a query, one retrieves one block following each of the indexes and combines the results. This approach achieves better recall than is achieved by having only one index. But in addition to the n times pre-processing overhead, it has to replicate the data $n - 1$ times to ensure that sequential IOs are possible via every index.

2.3 Clustering and Indexing by Clustering

Clustering techniques have been studied in the statistics, machine learning and database communities. The work in different communities focuses on different aspects of clustering. For instance, recent work in the database community includes CLARANS [36], BIRCH [50], DBSCAN [18], CLIQUE [1], and WaveClusters [41]. These techniques have high degree of success in identifying clusters in a very large dataset, but they do not deal with the efficiency of data search and data retrieval.

Of late, clustering techniques were explored for efficient indexing in high-dimensional spaces. Choubey, Chen and Rundensteiner [14] propose pre-processing data using clustering schemes before bulk-loading the clusters into an R-tree. They show that the bulk-loading scheme does not hurt retrieval efficiency compared to inserting tuples one at a time. Their bulk-loading scheme speeds up the inseration operation but is not designed to tackle the search efficiency problem that this study focuses on. For improving search efficiency, we presented the RIME system [13] that used Clindex [12, 32] for detecting replicated images. In this paper, we enhance our early work in two ways. First, we use SVMs for clustering. Second, we use not only centroids of clusters to locate nearby clusters but also support vectors, which register clusters' boundary information. Our experimental results show that SVMDex outperforms Clindex in both pre-processing efficiency and search accuracy. Bennett, Fayyad and Geiger [4] propose using the EM (Expectation Maximization) algorithm to cluster data for efficient data retrieval in high-dimensional spaces. The quality of the EM clusters, however, depends heavily on the initial setting of the model parameters and the number of clusters. When the data distribution does not follow a Gaussian distribution and the number of clusters and the Gaussian parameters are not initialized properly, the quality of the resulting clusters suffers. How these initial parameters should be selected is still an open research problem [7, 8]. For example, it is difficult to know

how many clusters exist before EM is run, but their EM algorithm needs this number to cluster effectively. Also, the EM algorithm may take many iterations to converge, and it may converge to one of many local optima [7, 34].

The clustering algorithm that SVMDex uses in this paper does not make any assumption on the number of clusters nor on the distribution of the data. Instead, we use a bottom-up approach that groups objects adjacent in the space into a cluster. (This is similar to grouping stars into galaxias.) Therefore, a cluster can be in any shape. We believe that when the data is not uniformly distributed, this approach can preserve the natural shapes of the clusters. In addition, SVMDex treats outliers differently by placing them in separate files. An outlier is not close to a cluster, so it is not similar to the objects in the clusters. However, an outlier can be close to other outliers. Placing outliers separately helps the search for the similar outliers more efficiently.

3 SVMDex Algorithm

Since the traditional approaches suffer from a large number of random IOs, our design objectives are (1) to reduce the number of IOs, and (2) to make the IOs sequential as much as possible. To accomplish these objectives, we propose a clustering/indexing approach. We call this scheme SVMDex (CLustering for INDEXing). The focus of this approach is to

- Cluster similar data on disk to minimize disk latency for retrieving similar objects, and
- Build an index on the clusters to minimize the cost of looking up a cluster.

To couple indexing with clustering, we propose using Support Vector Machines (SVMs). The remainder of this section is divided into four topics:

1. SVMs. Section 3.1 gives an overview on SVMs.
2. Clustering: Section 3.2 describes how SVMDex works in its clustering phase.
3. Indexing. Section 3.3 depicts how a structure is built by SVMDex to index clusters.
4. Search. Finally, we show how a similar query is conducted using SVMDex in Section 3.4.

3.1 Support Vector Machines

Support vector machines are a core machine learning technology. They have strong theoretical foundations and excellent empirical successes. They have been applied to tasks such as handwritten digit recognition [43], object recognition [37], and text classification [28].

We shall consider SVMs in the binary classification setting. We are given training data $\{\mathbf{x_1} \ldots \mathbf{x_n}\}$ that are vectors in some space $\mathcal{X} \subseteq \mathbb{R}^{\lceil}$. We are also given their labels $\{y_1 \ldots y_n\}$ where $y_i \in \{-1, 1\}$. In their simplest form, SVMs are hyperplanes that separate the training

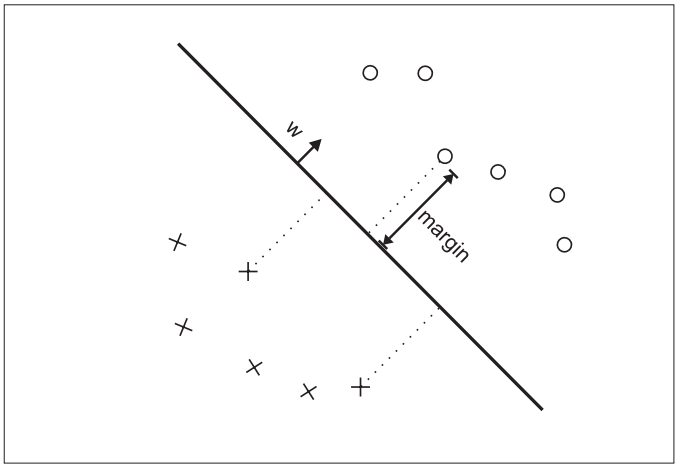


Figure 2: A simple linear Support Vector Machine

data by a maximal margin (see Fig. 2) . All vectors lying on one side of the hyperplane are labeled as -1, and all vectors lying on the other side are labeled as 1. The training instances that lie closest to the hyperplane are called *support vectors*. More generally, SVMs allow one to project the original training data in space $\mathcal{X}$ to a higher dimensional feature space $\mathcal{F}$ via a Mercer kernel operator K. In other words, we consider the set of classifiers of the form: $f(\mathbf{x}) = \sum_{\mathbf{i=1}}^{\mathbf{n}} \alpha_\mathbf{i} \mathbf{K}(\mathbf{x_i}, \mathbf{x})$. When $f(\mathbf{x}) \geq \mathbf{0}$ we classify $\mathbf{x}$ as $+1$, otherwise we classify $\mathbf{x}$ as -1.

When K satisfies Mercer's condition [10] we can write: $K(\mathbf{u}, \mathbf{v}) = \mathbf{\Phi}(\mathbf{u}) \cdot \mathbf{\Phi}(\mathbf{v})$ where $\Phi : \mathcal{X} \to \mathcal{F}$ and "$\cdot$" denotes an inner product. We can then rewrite f as:

$$f(\mathbf{x}) = \mathbf{w} \cdot \mathbf{\Phi}(\mathbf{x}), \text{ where } \mathbf{w} = \sum_{\mathbf{i=1}}^{\mathbf{n}} \alpha_\mathbf{i} \mathbf{\Phi}(\mathbf{x_i}). \tag{1}$$

Thus, by using K we are implicitly projecting the training data into a different (often higher dimensional) feature space $\mathcal{F}$. The SVM then computes the α_is that correspond to the maximal margin hyperplane in $\mathcal{F}$. By choosing different kernel functions we can implicitly project the training data from $\mathcal{X}$ into spaces $\mathcal{F}$ for which hyperplanes in $\mathcal{F}$ correspond to more complex decision boundaries in the original space $\mathcal{X}$.

Two commonly used kernels are the polynomial kernel $K(\mathbf{u}, \mathbf{v}) = (\mathbf{u} \cdot \mathbf{v} + \mathbf{1})^\mathbf{p}$ which induces polynomial boundaries of degree p in the original space $\mathcal{X}$ and the radial basis function kernel $K(\mathbf{u}, \mathbf{v}) = (\mathbf{e}^{-\gamma(\mathbf{u}-\mathbf{v})\cdot(\mathbf{u}-\mathbf{v})})$ which induces boundaries by placing weighted Gaussians upon key training instances. In the remainder of this paper we will assume that the modulus of the training data feature vectors are constant, i.e., for all training instances $\mathbf{x_i}$, $\|\Phi(\mathbf{x_i})\| = \lambda$ for some fixed λ. The quantity $\|\Phi(\mathbf{x_i})\|$ is always constant for radial basis function kernels, and so the assumption has no effect for this kernel. For $\|\Phi(\mathbf{x_i})\|$ to be constant with the polynomial kernels we require that $\|\mathbf{x_i}\|$ be constant. It is possible to relax this constraint on $\Phi(\mathbf{x_i})$.

3.2 Clustering Using SVMs

To perform efficient clustering in high-dimensional spaces, we follow three steps:

1. Projecting data to a higher dimensional space so that data can be separated by hyperplanes. For image data, we use Gaussian kernel function to perform this projection.

2. Computing distance between feature vectors in the projected space. We use cosine measure, which can be obtained by performing a dot product on two feature vectors.

3. Clustering vectors with close distance into a cluster. This step can be achieved efficiently using data mining algorithms that discover associations among items in an itemset. In our case, an item is a feature vector and association between two items is represented by their cosine distance. Our goal function is to find items that are weakly associated (i.e., that are near to each other).

3.3 Building Indexes

In the second step of the SVMDex algorithm, an indexing structure is built to support fast access to the clusters generated. All the objects in a cluster are stored in sequential blocks on disk, so that these objects can be retrieved by efficient sequential IOs. As shown in Figure 3, the indexing structure has a *cluster directory*. The cluster directory keeps track of information of each cluster, including its centroid, hyperplane, support vectors. To find the centroid, hyperplane, and support vectors a cluster, we conduct the following steps:

1. Computing the centroid of a cluster. This step is fairly straightforward. The centroid of a cluster is the average location of its member vectors.

2. Generating the hyperplane that separates the cluster from the rest of the data. Once clusters are generated, we can find the hyperplane that divides a cluster from the rest of the data by performing a binary classification. We label all vectors in the target cluster positive and the rest negative. SVMs generate an optimal hyperplane in the projected space to separate these two classes. For cluster j, its decision function can be written as

$$f_j(x) = \sum_{i=1}^{N} \alpha_i y_i K(x_i, x) + b,$$

where $x_i, i = 1, \ldots, N$ are feature values and K is the Mercer kernel.

3. Obtaining support vectors. The support vectors of a cluster are the vectors that are closest to the hyperplane. A standard SVM algorithm can produce support vectors after the optimal hyperplane is found.

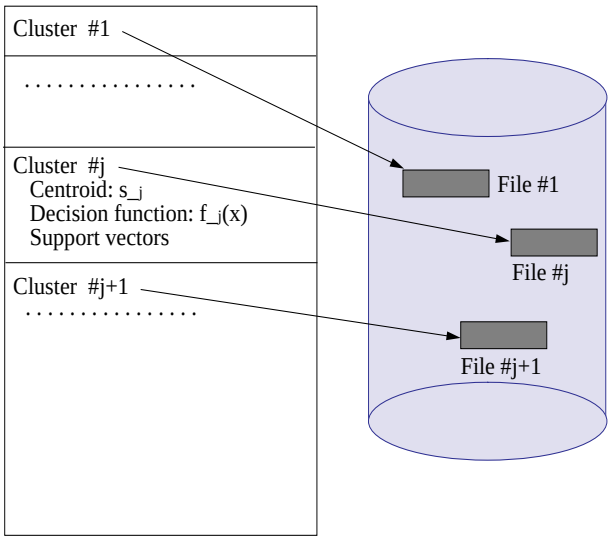


Figure 3: The index structure.

3.4 Similarity Search

Given a query object x, a similarity search is performed as follows:

1. For each cluster j, we compute $f_j(x)$

$$f_j(x) = \sum_{i=1}^{N} \alpha_i y_i K(x_i, x) + b,$$

2. For each cluster j, we also compute $s_j(x)$, the distance from the query object to the centroid of cluster j.

3. We characterize the distance between x and cluster j as

$$d_j(x) = f_j(x)/s_j(x).$$

bf 4. The closest cluster j^* to x has the largest positive $d_{j^*}(x)$ value.

If a high recall is desirable, we can read in more nearby clusters according to their $d_j(x)$ values. After we read candidate objects in these nearby clusters into main memory, we sort them according to their distances to the query object, and return the nearest objects to the user.

Remarks:

Our search can return more than one cluster whose centroid is close to the query object by checking the cluster directory. Since the number of clusters is much smaller than the number of objects in the dataset, the search for the nearest clusters can very likely be done via an in-memory lookup. If the number of clusters is very large, one may consider treating cluster

centroids as points and apply clustering algorithms on the centroids. This way, one can build a hierarchy of clusters and narrow down the search space to those clusters (in a group of clusters) that are nearest to the query object.

4 Evaluation

In our experiments we focused on queries of the form "find the top k most similar objects" or k Nearest Neighbors (abbreviated as k-NN). For each k-NN query, we return the top k nearest neighbors of the query object. To establish the benchmark to measure query performance, we scanned the entire dataset for each query object to find its top 20 nearest neighbors, the "golden" results. There are at least three metrics of interest to measure the query result:

(a) Recall after X IOs: After X IOs are performed, what fraction of the k top golden results has been retrieved?

(b) Precision after X IOs: After X IOs, what fraction of the objects retrieved is among the top k golden results?

(c) Precision after $R\%$ of the top k golden objects has been found. We call this *R-precision*, and it quantifies how much "useful" work was done in obtaining some fraction of the golden results.

In this paper we focus on recall results, and comment only briefly on R-precision and relative distance errors []. In our environment, we believe that precision is not the most useful metric since the main overhead is IOs, not the number of non-golden objects that are seen.

We performed our experiments on two sets of images: a $30,000$ and a $450,000$ image set. The first set contains $30,000$ images from Corel CDs, which has been extensively used as a test-bed of content-based image retrieval systems by the computer vision and image processing communities. This $30,000$-image dataset consists images of different categories, such as landscapes, portraits, and buildings. The other dataset is a set of $450,000$ randomly-crawled images from the Internet.[2] This $450,000$-image dataset has a variety of content, ranging from sports, cartoons, clip arts, etc. and can be difficult to classify even manually. To characterize images, we converted each image to a 48-dimensional feature vector by applying the Daubechies' wavelet transformation [13].

In addition to using the SVM clustering algorithm, we indexed these feature vectors using five other schemes: Equal Partition (EP), Tree-Structure Vector Quantization (TSVQ) [22], R*-tree, SR-tree, and M-tree with the PAC-NN scheme [15]:

[2]The readers are encouraged to experiment with the prototype, which is made available on-line [11]. We have been experimenting with our new approaches in image characterization so that the prototype may change from time to time.

- EP: To understand the role that clustering plays in SVMDex, we devised a simple scheme, EP, that partitions the dataset into sequential files without performing any clustering. That is, we partitioned the dataset into cells with an equal number of images, where each cell occupies a contiguous region in the space and is stored in a sequential file. Since EP is very similar to SVMDex except for the clustering, any performance differences between the schemes must be due to the clustering. If the differences are significant, it will mean that the dataset is not uniformly distributed and that SVMDex can exploit the clusters.
- TSVQ: To evaluate the effectiveness of SVMDex algorithm, we replaced it with a more sophisticated algorithm, and then stored the clusters sequentially as usual. The replacement algorithm used is TSVQ, a k-mean algorithm [35], was implemented for clustering data in high-dimensional spaces. It has been widely used in data compression and lately in indexing high-dimensional data for content-based image retrieval [44, 45].
- R*-tree and SR-tree: Tree structures are often used for similarity searches in multidimensional spaces. To compare, we used the R*-tree and SR-tree structures implemented by Katayama and Satoh [29]. We studied the IO cost and recall of using these tree-like structures to perform similarity search approximately. Note that the comparison between SVMDex and these tree-like structures will not be "fair" since neither R*-tree nor SR-tree performs off-line analysis (i.e., no clustering is done in advance). Thus, the results will only be useful to quantify the potential gains that the off-line analysis gives us, compared to traditional tree-based similarity search.
- M-tree with PAC-NN: We also compare SVMDex with Ciaccia and Patella's PAC-NN implementation of the M-tree [15]. The PAC-NN scheme uses two parameters, ϵ and δ, to adjust the tradeoff between search speed and search accuracy. The *accuracy* ϵ allows for a certain relative error in the results, and the confidence δ guarantees, with probability at least $(1-\delta)$, that ϵ will not be exceeded.

  Since the PAC-NN implementation that we use supports only 1-NN search, we performed 20 1-NN queries to get 20 nearest neighbors. After each 1-NN query, the nearest neighbor found was removed from the the M-tree. After the 20th 1-NN query, we had 20 nearest neighbors removed from the dataset and we compared them with the golden set to compute recall. Our test procedure is reliable for measuring recall for PAC-NN. However, our test procedure alters its search cost. It is well known that the cost of an exact 20-NN query is *not* 20 times the cost of a 1-NN query, but much less. For approximate queries, this seems also to be the case. To ensure fair comparison, we present the PAC-NN experimental results separately in Section 4.2.

As discussed in Section 3.4, SVMDex always retrieves the cluster whose centroid is closest to the query object. We also added this intelligence to both TSVQ and EP to improve their recall. For R*-tree and SR-tree, however, we did not add this optimization.[3]

[3] A scheme like R*-tree or SR-tree could be modified to pre-analyze the data and build a better structure. The results would be presumably similar to the results of EP, which keeps centroid information for each data

To measure recall, we used the cross-validation technique [35] commonly employed to evaluate clustering algorithms. We ran each test ten times, and each time we set aside 100 images as the test images and used the remaining images to build indexes. We then used these set-aside images to query the database built under four different index structures. We produced the average query recall for each run by averaging the recall of 100 image queries. We then took an average over 10 runs to obtain the final recall.

Our experiments are intended to answer the following questions:

1. How does clustering affect the recall of an approximate search? In Section 4.1 we perform different algorithms on two datasets that have different clustering quality to find out the effects of clustering.

2. How is the performance of SVMDex compared to the PAC-NN scheme? In Section 4.2 we show how the PAC-NN scheme trades accuracy for speed in an M-tree structure.

3. How is the performance of SVMDex in terms of elapsed time compared to the traditional structures and compared to sequentially reading in the entire file? In Section 4.3 we examine and compare how long the five schemes take to achieve a target recall.

4. How does block size affect recall? In Section 4.4 we cluster the $30,000$-image set using different block sizes to answer this question.

In the above experiments we collected the recall for 20-NN queries. In Section 4.5, we present the recalls of k-NN for $k = 1$ to 20.

4.1 Recall versus Clustering Algorithms

Figures 4 and 5 compare the recalls of SVMDex, TSVQ, EP, R*-tree and SR-tree. We discuss the experimental results on the two data sets separately.

- The $30,000$-image dataset (Figure 4): In this experiment, all schemes divided the dataset into about 256 clusters. Given one IO to perform, SVMDex returns the objects in the nearest cluster, which gives us an average of 62% recall (i.e., a return of 62% of the top 20 golden results). After we read three more clusters, the accumulated recall of SVMDex increases to 90%. If we read more clusters, the recall still increases but at a much slower pace. After we read 15 clusters, the recall approaches 100%. That is, we can selectively read in 6% ($\frac{15}{261} = 6\%$) of the data in the dataset to obtain almost all top 20 golden results.
  The EP scheme obtains much lower recall than SVMDex. It starts with 30% recall after the first IO is completed, and slowly progresses to 83% after 30 IOs. The TSVQ structure, although it does better than the EP scheme, still lags behind SVMDex. It obtains 35% recall after one IO, and the recall does not reach 90% until after 10 IOs. The recall of SVMDex and

block to assist effective nearest neighbor search. We did not develop the modified R*-tree or SR-tree scheme, and hence did not verify our hypothesis.

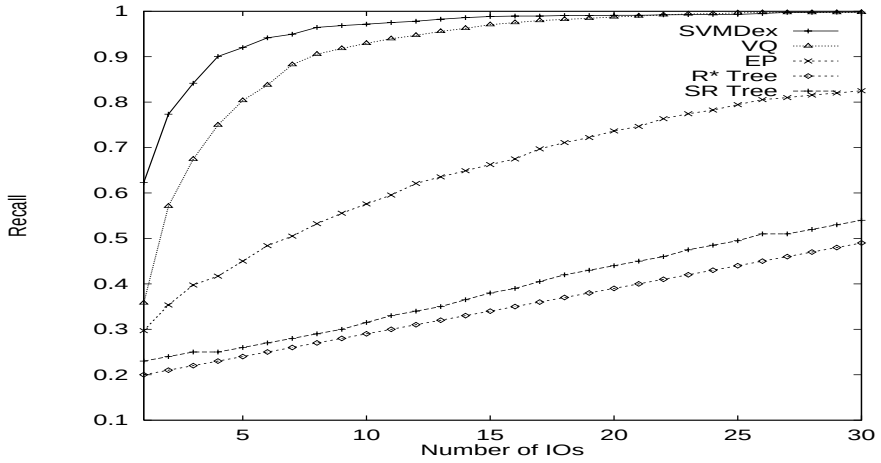


Figure 4: Recalls of Five Schemes (20-NN) on 30,000 Images.

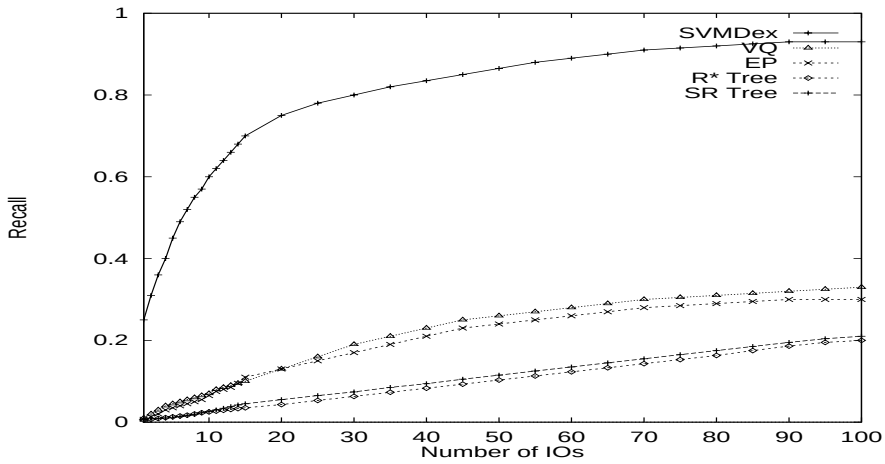


Figure 5: Recalls of Five Schemes (20-NN) on 450,000 Images.

TSVQ converge after 20 IOs. Finally, R*-tree and SR-tree suffer from the lowest recall. (That SR-tree performs better than R*-tree is consistent with the results reported in [29].)

- The 450,000-image dataset (Figure 5): In this experiment, all schemes divided the dataset into about 1,500 clusters. Given one IO to perform, SVMDex returns the objects in the nearest cluster, which gives us an average of 25% recall. After we read 10 more clusters, the accumulated recall of SVMDex increases to 60%. After we read 90 clusters, the recall approaches 90%. Again, we can selectively read in 6% ($\frac{90}{1500} = 6\%$) of the data in the dataset to obtain 90% of the top 20 golden results. As expected, the achieved recall of this image dataset is not as good as that of the 30,000-image set. We believe that this is because the randomly crawled images may not form clusters that are as well separated in the feature space as the clusters of the 30,000 image set. Nevertheless, in both cases the recall versus the

percentage of data retrieved from the data set shows that SVMDex can achieve a good recall by retrieving a small percentage of data.

The EP scheme achieves much lower recall than SVMDex. It starts with 1% recall after the first IO is completed, and slowly progresses to 33% after 100 IOs. The TSVQ structure performs worse than the EP scheme. It achieves only 30% recall after 100 IOs. Finally, R*-tree and SR-tree suffer from the lowest recall. When the data does not display good clustering quality, a poor clustering algorithm exacerbates the problem.

Remarks

Using recall alone cannot eitirely tell the quality of of the approximate results. For instance, at a given recall, say 60different (in terms of distances) from the query point and can be fairly similar to the query. Figure 6 uses the *relative distance error* metric proposed in [49] to report the quality of the approximation. Let Q denote the query point in the feature space, D_G^k the average distance of the top-k golden results from Q in the feature space, and D_A^k the average distance of the top-k actual results from Q. The relative distance error is measured by

$$\frac{D_A^k - D_G^k}{D_G^k}.$$

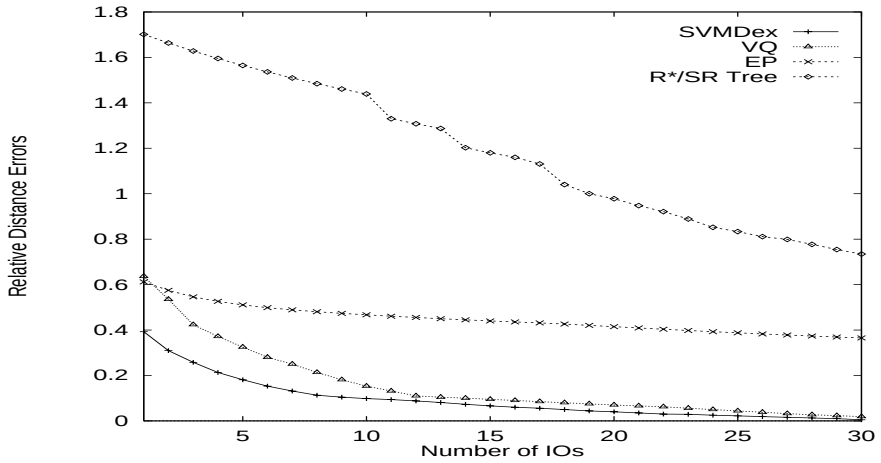


Figure 6: Relative Distance Errors (20-NN) on 30,000 Images.

Figure 6 shows that the relative distance errors of both SVMDex and TSVQ can be kept under 20% after ten IOs. But the relative distance errors of the tree-structures are still quite large (more than 80%) even after 30 IOs.

4.2 The PAC-NN Scheme

We conducted PAC-NN experiments on the 30,000-image dataset. We observed that the value of δ plays a more important role than the value of ϵ in determining the search quality and cost. Table 1 shows one representative set of results where we set $\epsilon = 0.2$. (Setting ϵ to other values does not change the tradeoff pattern between search accuracy and cost.) Under five different δ settings, 0, 0.01, 0.02, 0.05 and 0.1, the table shows recall, number of IOs and wall-clock query time (for all 20 1-NN queries). (As we mentioned previously, the cost of 20 1-NN queries can be substantially higher than the cost of one 20-NN query. Therefore, one should only treat the number of IOs and the elapsed time in the table as loose bounds of the cost.)

When δ is increased from zero to 0.01, the recall of PAC-NN drops from 95.3% to 40%, but the number of IOs is reduced substantially to one eighth and the wall-clock elapsed time is reduced to one fifth. When we increase δ further, both the recall and cost continue to drop.

Remarks

- Quality of approximation: From the perspective of some other performance metrics that were proposed to measure quality of approximation [49], the PAC-NN scheme trades very slight accuracy for substantial search speedup. For multimedia applications where one may care more about getting similar results than finding the exact top-NN results, the PAC-NN scheme provides a scalable and effective way to trade accuracy for speed.
- IO cost: Since the PAC-NN scheme we experimented with is implemented with M-tree and a tree-like structure tends to distribute a nature cluster into many non-contiguous disk blocks (we have discussed this problem in Section 2), it takes the M-tree more IOs than SVMDex to achieve a given recall. This observation is consistent with the results obtained from using other tree-like structures to index the images.
- Construction Time: It took only around 1.5 minutes to build an M-tree, while building SVMDex with SVMDex takes 30 minutes on a 600MHz Pentium-III workstation with 512 MBytes of DRAM. As we explained in Section 3, since the clustering phase is done off-line, the pre-processing time may be acceptable in light of the speed that is gained in query performance.

	$\delta = 0$	$\delta = 0.01$	$\delta = 0.02$	$\delta = 0.05$	$\delta = 0.1$
Recall	95.3%	40.0%	25.3%	12.2%	5.9%
Number of IOs	13,050	1,580	1,200	933	800
Wall-clock Time (seconds)	15	3	2	1.6	1.3

Table 1: The Performance and Cost of PAC-NN.

Parameter Name	*Value*
Disk Capacity	*26 GBytes*
Number of cylinders, CYL	*50,298*
Min. Transfer Rate TR	*130 Mbps*
Rotational Speed (RPM)	*5,400*
Full Rotational Latency Time	*11.12 ms*
Min. Seek Time	*2.0 ms*
Average Seek Time	*8.9 ms*
Max. Seek Time	*18 ms*

Figure 7: Quantum Fireball Lct Disk Parameters

4.3 Running Time versus Recall

We have shown that clustering indeed helps improve recall when a dataset is not uniformly distributed in the space. This is shown by the recall gap between SVMDex and EP in Figures 4 and 5. We have also shown that by retrieving only a small fraction of data, SVMDex can achieve a recall that is quite high (e.g., 90%). This section shows the time needed by the schemes to achieve a target recall. We compare their elapsed time to the time it takes to sequentially read in the entire dataset.

To report query time, we first use a quantitative model to compute IO time. Since IO time dominates a query time, this model helps explain why one scheme is faster than the others. We then compare the wall-clock time we obtained with the computed IO time. As we will show shortly, these two methods give us different elapsed times but the relative performance between the indexes are the same.

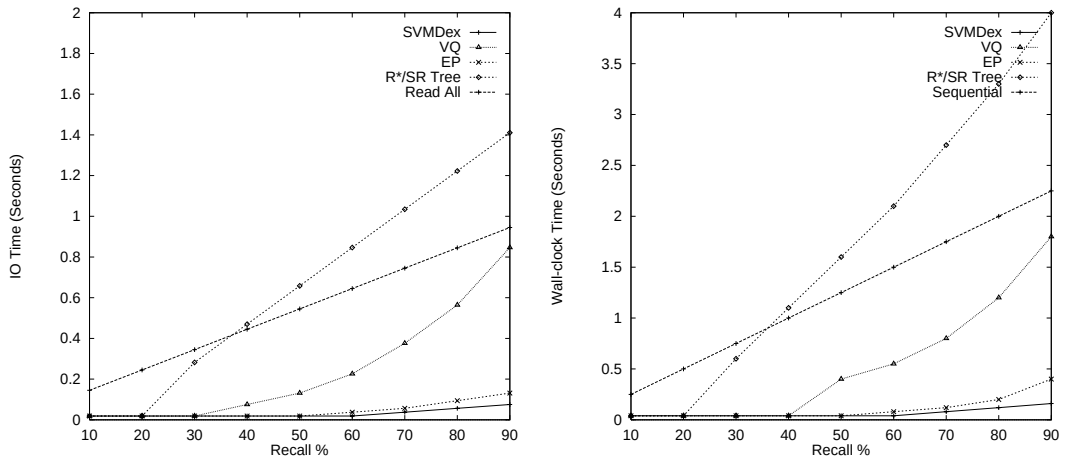
Let B denote the total amount of data retrieved to achieve a target recall, N the number of IOs, TR the transfer rate of the disk, and T_{seek} the average disk latency. The elapsed time T of a search can be estimated as

$$T = N \times T_{seek} + B/TR.$$

To compute T, we use the parameters of a modern disk listed in Table 7. To simplify the computation, we assume that an IO incurs one average seek (8.9 ms) and one half of a rotational delay (5.6 ms). We also assume that TR is 130 Mbps and that each image record, including its wavelets and a URL address, takes up 500 Bytes. For example, to compute the elapsed time to retrieve 4 clusters that contains 10% of the 30,000 images, we can express T as

$$T = 4 \times 14.5 + \frac{30,000 \times 10\% \times 500 \times 8}{130}\ ms.$$

Figure 8(a) shows the average elapsed time versus recall for five schemes on the 30,000-image dataset. Note that the average elapsed time is estimated based on the number of records



(a) IO Time (b) Wall-clock Time

Figure 8: Elapsed Time vs. Recall on 30,000 images.

retrieved in our ten rounds of experiments, not based on the number of clusters retrieved. The horizontal line in the middle of the figure shows the time it takes to read in the entire image feature file sequentially. To achieve 90% recall, SVMDex and TSVQ take substantially less time than reading in the entire file. The traditional tree structures, on the other hand, perform worse than the sequential scan when we would like to achieve a recall that is above 65%.

Figure 8(b) shows the wall-clock elapsed time versus recall that we collected on a Dell workstation, which is configured with a 600MHz Pentium-III processor and 512 MBytes of DRAM. To eliminate the effect of caching, we ran ten queries on each index structure and we rebooted the system after each query. The average running time of all schemes are about twice as long as the predicted IO time. We believe that this is because the wall-clock time includes index lookup time, distance computation time, IO bus time and memory access time, in addition to the IO time. The relative performance between indexes, however, are unchanged. In both figures, SVMDex with SVMDex requires the lowest running time to reach every recall target.

In short, we can draw the following conclusions:

1. SVMDex achieves a higher recall than TSVQ, EP and R*-tree, because SVMDex is adaptive to wide variances of cluster shapes and sizes. TSVQ and other algorithms are forced to bound a cluster by linear functions, e.g., a cube in 3-D space. SVMDex is not constrained in this way. Thus, if we have an odd cluster shape (e.g., with tentacles), SVMDex will conform to the shape, while TSVQ will either have to pick a big space that encompasses all the "tentacles," or will have to select several clusters, each for a portion of the

shape, and will thereby inadvertently break some "natural" clusters, as we illustrated in Figure 1(a). It is not surprising that the recall of TSVQ converges to that of SVMDex after a number of IOs, because TSVQ eventually pieces together its "broken" clusters.

2. Using SVMDex to approximate an exact similarity search requires reading just a fraction of the entire dataset. The performance is far better than that achieved by performing a sequential scan on the entire dataset.

3. SVMDex, TSVQ and EP enjoy a boost in recall in their first few additional IOs, since they always retrieve the next cluster whose centroid is the closest to the query object. R*-tree, conversely, does not store centroid information, and cannot selectively retrieve blocks to boost its recall in the first few IOs. (We have discussed the shortcomings of tree structures in Section 2 in detail.)

4. When a data set does not exhibit good clusters, SVMDex's effectiveness can suffer. A poor clustering algorithm such as TSVQ and EP can suffer even more. Thus, employing a good clustering algorithm is important for SVMDex to work well.

	Recall	60%	70%	80%	90%	≈ 100%
	# of Golden Objects Retrieved	12	14	16	18	20
SVMDex	*# of Objects Retrieved*	115	230	345	460	1,725
	R-Precision	10.44%	6.09%	4.64%	3.91%	1.16%
R-tree*	*# of Objects Retrieved*	2,670	3,430	4,090	4,750	5,310
	R-Precision	0.45%	0.41%	0.39%	0.379%	0.377%

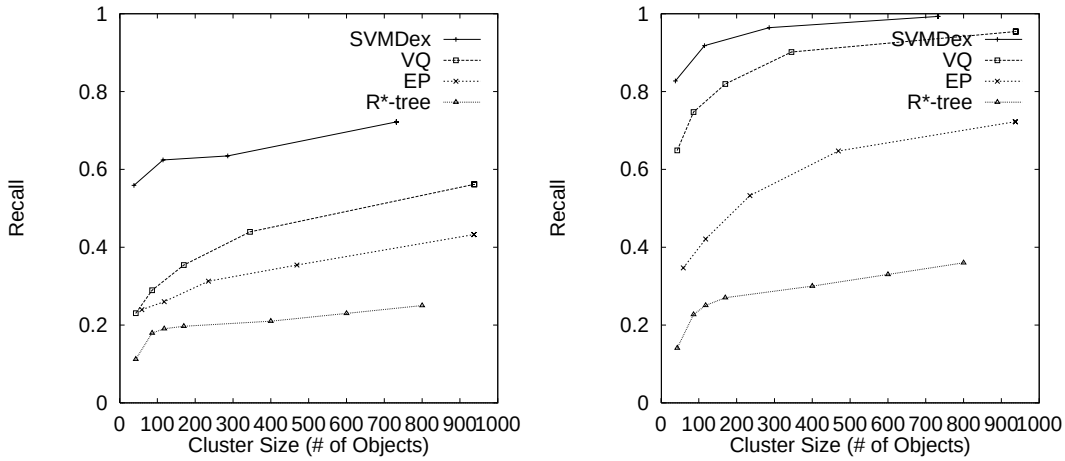
Table 2: R-precision of 20-NN

Remarks on Precision

Figure 4 shows that while retrieving the same amount of data, SVMDex has higher recall than R*-tree and other schemes. Put another way, SVMDex also enjoys higher R-precision because retrieving the same number of objects from disk gives SVMDex more golden results.

Since a tree structure is typically designed to use a small block size to achieve high precision, we tested R*-tree method with larger block size, and compared its R-precision with that of SVMDex. We conducted this experiment only on the 30,000-image dataset. (The tree-like structures are ineffective on the 450,000-image dataset.) It took R*-tree 267 IOs on average to complete an exact similarity search. Table 2 shows that the R-precision of SVMDex is more than ten times higher than that of R*-tree under different recall values.

This result, although surprising, is consistent with that of many studies [29, 46]. The common finding is that when the data dimension is high, tree structures fail to divide points into neighborhoods and are forced to access almost all leaves during an exact similarity search.



(a) After one IO

(b) After five IOs

Figure 9: Recall versus Cluster Size.

Therefore, the argument for using a small block size to improve precision becomes weaker as the data dimension increases.

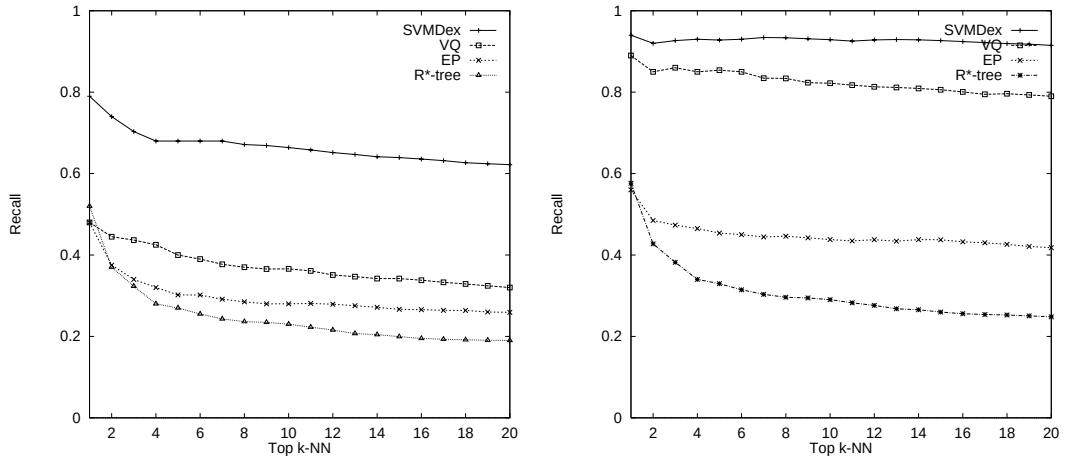
4.4 Recall versus Cluster Size

Since TSVQ, EP and tree-like structures are ineffective on the 450,000-image dataset, we conducted the remaining experiments on the 30,000-image dataset only.

We define cluster size as the average number of objects in a cluster. To see the effects of the cluster size on recall, we collected recall values at different cluster sizes for SVMDex, TSVQ, EP and R*-tree. Note that the cluster size of SVMDex is determined by the selected values of κ and θ. We thus set four different κ and θ combinations to obtain recall values for four different cluster sizes. For EP, TSVQ and R*-tree, we selected cluster (block) sizes that contain from 50 up to 900 objects. Figure 9 depicts the recall (y-axis) of the four schemes for different cluster sizes (x-axis) after one IO and after five IOs are performed.

Figure 9(a) shows that given the same cluster size, SVMDex has a higher recall than TSVQ, EP and R*-tree. The gaps between the clustering schemes (EP and R*-tree) and non-clustering schemes (TSVQ and SVMDex) widen as the cluster size increases. We believe that the larger the cluster size, the more important a role the quality of the clustering algorithm plays. SVMDex enjoys significantly higher recall than TSVQ, EP and R*-tree because it captures the clusters better than the other three schemes.

Figure 9(b) shows that SVMDex still outperforms TSVQ, EP and R*-tree after five IOs. SVMDex, TSVQ and EP enjoy better improvement in recall than R*-tree because, again, SVMDex, TSVQ and EP always retrieve the next cluster whose centroid is nearest to the



(a) After One IO (b) After Five IOs

Figure 10: Recall of k-NN.

query object. On the other hand, a tree structure does not keep the centroid information, and the search in the neighborhood can be random and thus suboptimal. We believe that adding centroid information to the leaves of the tree structures can improve its recall.

4.5 Relative Recall versus k-NN

So far we measured only the recall of returning the top 20 nearest neighbors. In this section we present the recall when the top $k = 1$ to 19 nearest neighbors are returned after one and five IOs are performed. In these experiments, we partitioned the dataset into about 256 clusters (blocks) for all schemes, i.e., all schemes have about the same cluster size.

Figures 10(a) and Figures 10(b) present the relative recall relative to the top 20 golden results of the four schemes after one IO and five IOs are performed, respectively. The x-axis in the figures represents the number of nearest neighbors requested, and the y-axis represents the relative recall. For instance, when only one nearest object is requested, SVMDex returns the nearest object 79% of the time after one IO and 95% of the time after five IOs.

Both figures show that the recall gaps between schemes are insensitive to how many nearest neighbors are requested. We believe that once the nearest object can be found in the returned cluster(s), the conditional probability that additional nearest neighbors can also be found in the retrieved cluster(s) is high. Of course, for this conclusion to hold, the number of objects contained in a cluster must be larger than the number of nearest neighbors requested. In our experiments, a cluster contains 115 objects on average, and we tested up to 20 golden results. This leads us to the following discussion on how to tune SVMDex's control parameters to form clusters of a proper size.

4.6 Summary

In summary, our experimental results show that:

1: Employing a better clustering algorithm improves recall as well as the elapsed time to achieve a target recall.

2: Providing additional information such as the centroids of the clusters helps prioritize the retrieval order of the clusters, and hence improves the search efficiency.

3: Using a large block size is good for both IO efficiency and recall.

Our experimental results also show that SVMDex is effective when it uses a good clustering algorithm and when the data is not uniformly distributed. When a dataset is uniformly distributed in a high-dimensional space, sequentially reading in the entire dataset can be more effective than using SVMDex.

5 Conclusions and Relationship to Other Patent Documents

In this paper, we presented SVMDex, a paradigm for performing approximate similarity search in high-dimensional spaces to avoid the dimensionality curse. SVMDex clusters similar objects on disk, and performs a similarity search by finding the clusters near the query object. This approach improves the IO efficiency by clustering and retrieving relevant information sequentially on and from the disk. We believe that for many high-dimensional datasets (e.g., documents and images) that exhibit clustering patterns, SVMDex is an attractive approach to support approximate similarity search both efficiently and effectively.

Experiments showed that SVMDex typically can achieve 90% recall after retrieving a small fraction of data. SVMDex's recall is much higher than that of some traditional index structures. Through experiments we also learned that SVMDex works well because it uses large blocks, finds clusters more effectively, and searches the neighborhood more intelligently by using the centroid and support vector information.

SVMDex helps sampling and retrieval efficiency of MEGA (Patent Document #2) and SVM_{Active} (Patent Document #3).

References

[1] R. Agrawal, J. Gehrke, D. Gunopulos, and P. Raghavan. Automatic subspace clustering of high dimensional data for data mining applications. *Proceedings of ACM SIGMOD*, June 1998.

[2] S. Arya, D. Mount, N. Netanyahu, R. Silverman, and A. Wu. An optimal algorithm for approximate nearest neighbor searching in fixed dimensions. *Proceedings of the 5th SODA*, pages 573–82, 1994.

[3] N. Beckmann, H.-P. Kriegel, R. Schneider, and B. Seeger. The R*-tree: an efficient and robust access method for points and rectangles. *Proceedings of ACM SIGMOD*, May 1990.

[4] K. P. Bennett, U. Fayyad, and D. Geiger. Density-based indexing for approximate nearest-neighbor queries. *KDD*, 1999.
[5] S. Berchtold, C. Bohm, and H.-P. Kriegel. The pyramid-technique: Towards breaking the curse of dimensionality. *ACM Sigmod*, May 1998.
[6] S. Berchtold, D. A. Keim, and H.-P. Kriegel. The X-Tree: An index structure for high-dimensional data. *Proceedings of the 22nd VLDB*, August 1996.
[7] C. Bishop. *Neural Networks for Pattern Recognition*. Oxford, 1998.
[8] P. S. Bradley, U. M. Fayyad, and C. A. Reina. Scaling em (expectation-maximization) clustering to large databases. *Microsoft Technical Report MSR-TR-98-34*, November 1999.
[9] S. Brin and H. Garcia-Molina. Copy detection mechanisms for digital documents. *Proceedings of ACM SIGMOD*, May 1995.
[10] C. Burges. A tutorial on support vector machines for pattern recognition. *Data Mining and Knowledge Discovery*, 2:121–167, 1998.
[11] E. Chang and T. Cheng. Perception-based image retrieval. *ACM Sigmod (Demo)*, May 2001.
[12] E. Chang, C. Li, J. Wang, P. Mork, and G. Wiederhold. Searching near-replicas of images via clustering. *Proc. of SPIE Symposium of Voice, Video, and Data Communications*, September 1999.
[13] E. Chang, J. Wang, C. Li, and G. Wiederhold. RIME - a replicated image detector for the www. *Proc. of SPIE Symposium of Voice, Video, and Data Communications*, November 1998.
[14] R. Choubey, L. Chen, and E. A. Rundensteiner. Gbi: A geberalized r-tree bulk-insertion. *Proceedings of the* 8th *SSD*, pages 91–108, July 1999.
[15] P. Ciaccia and M. Patella. Pac nearest neighbor queries: Approximate and controlled search in high-dimensional and metric spaces. *Proceedings of ICDE*, pages 244–255, 2000.
[16] P. Ciaccia, M. Patella, and P. Zezula. M-Tree: An efficient access method for similarity search in metric spaces. *Proceedings of the 23rd VLDB*, August 1997.
[17] K. Clarkson. An algorithm for approximate closest-point queries. *Proceedings of the 10th SCG*, pages 160–64, 1994.
[18] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. *Proceedings of the 2nd International Conference on Knowledge Discovery in Databases and Data Mining*, August 1996.
[19] M. Flickner, H. Sawhney, J. Ashley, Q. Huang, B. Dom, M. Gorkani, J. Hafner, D. Lee, D. Petkovic, D. Steele, and P. Yanker. Query by image and video content: the QBIC system. *IEEE Computer*, 28(9):23–32, 1995.
[20] H. Garcia-Molina, S. Ketchpel, and N. Shivakumar. Safeguarding and charging for information on the internet. *Proceedings of ICDE*, 1998.
[21] H. Garcia-Molina, J. Ullman, and J. Widom. *Database System Implementation*. Prentice Hall, 1999.
[22] A. Gersho and R. Gray. *Vector Quantization and Signal Compression*. Kluwer Academic, 1991.
[23] A. Gupta and R. Jain. Visual information retrieval. *Comm. of the ACM*, 40(5):69–79, 1997.
[24] A. Guttman. R-trees: a dynamic index structure for spatial searching. *Proc. of ACM SIGMOD*, June 1984.
[25] G. R. Hjaltason and H. Samet. Ranking in spatial databases. *Proceedings of the* 4th *SSD*, pages 83–95, August 1995.
[26] P. Indyk, A. Gionis, and R. Motwani. Similarity search in high dimensions via hashing. *Proceedings of the 25th VLDB*, September 1999.
[27] P. Indyk and R. Motwani. Approximate nearest neighbors: Towards removing the curse of dimensionality. *Proceedings of the 30th STOC*, pages 604–13, 1998.
[28] T. Joachims. Text categorization with support vector machines. In *Proceedings of the European Conference on Machine Learning*. Springer-Verlag, 1998.
[29] N. Katayama and S. Satoh. The SR-Tree: An index structure for high-dimensional nearest neighbor queries. *Proceedings of ACM SIGMOD*, May 1997.
[30] J. M. Kleinberg. Two algorithms for nearest-neighbor search in high dimensions. *Proceedings of the 29th STOC*, 1997.
[31] E. Kushilevitz, R. Ostrovsky, and Y. Rabani. Efficient search for approximate nearest neighbor in high dimensional spaces. *Proceedings of the 30th STOC*, pages 614–23, 1998.

[32] C. Li, E. Chang, H. Garcia-Molina, and G. Wiederhold. Clustering for approximate similarity queries in high-dimensional spaces. *IEEE Transaction on Knowledge and Data Engineering (to appear)*, 2001.

[33] K.-L. Lin, H. V. Jagadish, and C. Faloutsos. The TV-tree: an index structure for high-dimensional data. *VLDB Journal*, 3(4), 1994.

[34] G. J. McLachlan and T. Krishnan. *The EM algorithm & extensions*. John Wiley & Sons, 1997.

[35] T. M. Mitchell. *Machine Learning*. McGraw-Hill, 1997.

[36] R. T. Ng and J. Han. Efficient and effective clustering methods for spatial data mining. *Proceedings of the 20th VLDB*, September 1994.

[37] C. Papageorgiou, M. Oren, and T. Poggio. A general framework for object detection. In *Proceedings of the International Conference on Computer Vision*, 1998.

[38] J. T. Robinson. The K-D-B-Tree: A search structure for large multidimensional dynamic indexes. *Proceedings of ACM SIGMOD*, April 1981.

[39] N. Roussopoulos, S. Kelley, and F. Vincent. Nearest neighbor queries. *ACM Sigmod*, pages 71–79, 1995.

[40] S. Santini and R. Jain. Similarity matching. *IEEE Transaction on Pattern Analysis and Machine Intelligence*, 21(9):871–883, September 1999.

[41] G. Sheikholeslami, S. Chatterjee, and A. Zhang. Wavecluster: A multi-resolution clustering approach for very large spatial databases. *Proceedings of the 34th VLDB Conference*, August 1998.

[42] J. R. Smith and S.-F. Chang. Visualseek: A fully automated content-based image query system. *ACM Multimedia Conference*, 1996.

[43] V. Vapnik. *Statistical Learning Theory*. Wiley, 1998.

[44] J. Z. Wang, G. Wiederhold, O. Firschein, and S. X. Wei. Wavelet-based image indexing techniques with partial sketch retrieval capability. *Proceedings of the 4th ADL*, May 1997.

[45] J. Z. Wang, G. Wiederhold, O. Firschein, and S. X. Wei. Content-based image indexing and searching using daubechies' wavelets. *Journal of Digital Libraries*, 1(4):311–28, 1998.

[46] R. Weber, H. Schek, and S. Blott. A quantitative analysis and performance study for similarity-search methods in high-dimensional spaces. *Proceedings of the 24th VLDB*, pages 194–205, 1998.

[47] D. A. White and R. Jain. Similarity indexing: Algorithms and performance. *Proc. SPIE Vol.2670, San Diego*, 1996.

[48] D. A. White and R. Jain. Similarity indexing with the SS-Tree. *Proceedings of the 12th ICDE*, Feb. 1996.

[49] P. Zezula, P. Savino, G. Amato, and F. Rabitti. Approximate similarity retrieval with m-trees. *VLDB Journal*, 7(4):275–293, December 1998.

[50] T. Zhang, R. Ramakrishnan, and M. Livny. Birch: An efficient data clustering method for very large databases. *Proceedings of ACM SIGMOD*, June 1996.